

Шпаргалка по Git

=====

- * [Установка и настройка git](#установка-и-настройка-git)
 - * [Псевдокоманды](#псевдокоманды)
- * [Создание репозитория](#создание-репозитория)
 - * [Создание нового пустого репозитория](#создание-нового-пустого-репозитория)
 - * [Создание клона существующего репозитория](#создание-клона-существующего-репозитория)
- * [Команды просмотра](#команды-просмотра)
- * [Создание коммита](#создание-коммита)
 - * [Сохранение невошедших в коммиты изменений (зачека)](#сохранение-невошедших-в-коммиты-изменений-зачека)
- * [Переходы к коммитам](#переходы-к-коммитам)
- * [Ветки](#ветки)
- * [Редактирование веток](#редактирование-веток)
- * [Работа с удалённым репозиторием](#работа-с-удалённым-репозиторием)
- * [Метки](#метки)
- * [История изменений](#история-изменений)

Git - это система контроля версий, созданная Линусом Торвальдсом (создателем Linux) для

Репозиторий Git находится в каталоге `.git`, который содержит все данные (в виде собственн
При работе с общим кодом пользователь git владеет своим отдельным репозиторием, в которо

Команды git обычно имеют вид ``git команда опции``. Их описания доступны в `man`, используя

Установка и настройка git</h1>

Установка git для Linux:

...

```
# apt-get install git
```

Установка git для Windows:

Если собираетесь использовать для git текстовый редактор из Visual Studio, установите её

Скачайте дистрибутив Git (и желаемые GUI) для своей версии Windows [через git-scm.com](https://git-scm.com)

После установки в списке программ появится пункт 'Git Bash', запускающий терминал, в кот

Настройка git:

Если не задать для git имя и почту пользователя для заголовков коммитов, при каждом комм

...

```
$ git config --global user.name <имя автора коммитов для git>
$ git config --global user.email <e-mail автора коммитов для git>
```

Задать желаемый текстовый редактор для редактирования описаний изменений:

...

```
$ git config --global core.editor "<команда для вызова текстового редактора, не возвраща
```

Задать режим разрешения конфликтов с отображением первоначального состояния по-разному о

...

```
$ git config --global merge.conflictstyle diff3
```

По умолчанию отправлять изменения в ту ветку другого репозитория, с которой связана лока

...

```
$ git config --global push.default simple
...
```

Разрешить повторное использование результатов разрешения конфликтов:

```
...
$ git config --global rerere.enabled true
...
```

Можно также настроить команду `'git log'` для постоянного вывода известных ссылок на ком

```
...
$ git config --global log.decorate full
...
```

То, на какие файлы git обращает внимание по умолчанию (например, при показе состояния он

Псевдокоманды

Git позволяет создавать alias-ы, и упростить использование некоторых длинных команд.

Например, создав alias:

```
...
$ git config --global alias.log-graph 'log --graph --decorate=full --oneline'
...
```

в дальнейшем можно вызывать log с этими опциями так:

```
...
$ git log-graph HEAD master
...
```

Некоторые полезные alias-ы:

```
...
$ git config --global alias.s 'status'
$ git config --global alias.bvv 'branch -vv'
$ git config --global alias.logl 'log --graph --decorate=full --oneline'
$ git config --global alias.logo 'log --graph --decorate=full --format=short'
$ git config --global alias.logg 'log --graph --stat --decorate=full'
$ git config --global alias.logf 'log --graph --stat --decorate=full --format=fuller'

$ git config --global alias.diffu 'diff @{u}'
$ git config --global alias.rebaseu 'rebase -i @{u}'
$ git config --global alias.diffb '!git diff `git merge-base HEAD origin/master`'
$ git config --global alias.rebaseb '!git rebase -i `git merge-base HEAD origin/master`'
$ git config --global alias.diffm 'diff origin/master'
$ git config --global alias.rebasem 'rebase -p origin/master'
$ git config --global alias.push-force 'push --force-with-lease'
```

...

Создание репозитория

Создание нового пустого репозитория

Если каталог будущего репозитория ещё не существует, создайте его:

```
...
$ mkdir <верхний каталог будущего репозитория>
...
```

Перейдите в каталог будущего репозитория и создайте репозиторий git:

```
...
$ cd <верхний каталог будущего репозитория>
$ git init
...
```

Сразу после создания нового пустого репозитория мы находимся на ветке master (на самом д

Создание клона существующего репозито

Создание клона существующего репозитория (текущей будет та ветка, которая была текущей в

...

```
$ git clone <URL каталога, содержащего .git> [путь_к_новому_репозиторию]
```

...

По умолчанию исходный репозиторий будет записан в настройках клона с именем “origin”.

Команды просмотра</h1>

Показать текущее состояние репозитория (самая часто используемая команда git):

...

```
$ git status
```

...

Показать историю коммитов (--stat показывает изменённые файлы со статистикой сделанных и

...

```
$ git log [--stat] [<(коммит)>] [<имена файлов или директорий>]
```

...

Показать граф отношения веток, сократив оглавления коммитов до одной строки:

...

```
$ git log --graph --decorate=full --oneline [<(коммит 1)>] [<(коммит 2)>] ...
```

...

Показать разницу файлов (если второй коммит не указан, сравнивается текущее состояние от

...

```
$ git diff [<(исходный коммит)> [<(сравниваемый коммит)>]] [<имена файлов или директорий>]
```

...

Создание коммита</h1>

Коммит – это зафиксированное в репозитории состояние всех отслеживаемых файлов. Каждый к
В командах, в которых нужно обратиться к коммиту, к нему можно обращаться как указывая в

Специальное имя HEAD содержит идентификатор текущего коммита или имя текущей ветки.

Здесь и далее там, где можно указать идентификатор коммита, имя ветки, имя тэга или HEAD

Добавление файлов в индекс для коммита:

...

```
$ git add <имена файлов и директорий>
```

...

При добавлении файлов в индекс для коммита они не помечаются, а “копируются”. Если после

Удаление файлов из индекса для коммита:

...

```
$ git reset <имена файлов и директорий>
```

...

Удаление файлов из каталога и добавление удаления из коммита в индекс для коммита:

...

```
$ git rm <имена файлов>
$ git rm -r <имена файлов и директорий>
...
```

Переименование файлов в каталоге и добавление переименования в коммите в индекс для комм

...

```
$ git mv <старое имя> <новое имя>
$ git mv <старое имя>... <директория>
...
```

Создать коммит (файлы в индексе для коммита будут добавлены в репозиторий, текущий комм

...

```
$ git commit
...
```

При создании коммита вызывается текстовый редактор, в котором необходимо создать описани
Можно также передать описание в командной строке:

...

```
$ git commit -m "<описание>"
...
```

При добавлении коммита его идентификатор записывается в HEAD или в текущую ветку.

Редактировать текущий коммит (файлы в индексе для коммита будут добавлены в репозиторий,

...

```
$ git commit --amend
...
```

Создать коммит, помеченный как исправление ранее сделанного коммита:

...

```
$ git commit --fixup=<(исправляемый коммит)>
...
```

[Сохранение невошедших в коммиты изменений-зачка](#)Сохранение невошедших в комм

Приведение отслеживаемых файлов к состоянию, записанному в текущем коммите, с сохранение

...

```
$ git stash
...
```

Восстановление сохранённых не добавленных в коммиты изменений:

...

```
$ git stash apply
...
```

Удаление сохранённых не добавленных в коммиты изменений из git:

...

```
$ git stash clear
...
```

[Переходы к коммитам](#)Переходы к коммитам

Приведение отслеживаемых файлов к состоянию, записанному в коммите, с учётом не добавлен

...

```
$ git checkout <(коммит)>
...
```

Приведение выбранных файлов к состоянию, записанному в коммите, без учёта не добавленных

```
...
$ git checkout <(коммит)> <имена файлов и директорий>
...
```

Приведение отслеживаемых файлов к состоянию, записанному в коммите, с потерей не добавле

```
...
$ git checkout -f [<(коммит)>]
...
```

Ветки

Коммиты организованы в последовательности (ветки). Имя ветки – это заданное пользователе

Список всех веток:

```
...
$ git branch -a
...
```

Список локальных веток:

```
...
$ git branch
...
```

Список веток с их базовыми ветками и последними коммитами в них:

```
...
$ git branch -vv
...
```

Показать все ветки, содержащие в названии слово “vector”:

```
...
$ git branch --list -a “*vector*”
...
```

Переход к ветке:

```
...
$ git checkout <имя_ветки>
...
```

Создание ветки для текущего коммита (имя ветки это имя файла, оно не должно содержать пр

```
...
$ git checkout -b <имя_новой_ветки>
...
```

Создание ветки для произвольного коммита (текущая ветка не меняется):

```
...
$ git branch <имя_новой_ветки> [<(коммит)>]
...
```

Переместить текущую ветку на указанный коммит (коммиты в ветке после указанного могут бы

```
...
$ git reset <(коммит)>
...
```

Переименование ветки:

```
...  
$ git branch -m <старое_имя_ветки> <новое_имя_ветки>  
...
```

Удаление ветки без потери информации:

```
...  
$ git branch -d <имя_ветки>  
...
```

Удаление ветки с возможной потерей информации:

```
...  
$ git branch -D <имя_ветки>  
...
```

<h1 id="#редактирование-веток">Редактирование веток</h1>

В личной ветке можно редактировать историю коммитов (никогда не редактируйте коммиты, уже созданные в общем репозитории).
“Перебазирование”, текущий коммит и его предшественники, отличающиеся от указанного коммита.

```
...  
$ git rebase -i <(коммит)>  
...
```

Назначить базовую ветку для рабочей ветки (она будет доступна по имени "@{u}"):

```
...  
$ git branch <имя_рабочей_ветки> -u <имя_базовой_ветки>  
...
```

Перебазирование текущей ветки от соответствующей ей базовой ветки:

```
...  
$ git rebase -i @{u}  
...
```

Перебазирование текущей ветки от главной ветки общего репозитория (обычно origin/master)

```
...  
$ git rebase -p origin/master  
...
```

Создание коммита, отменяющего изменения, сделанные в перечисленных коммитах:

```
...  
$ git revert <коммит>...  
...
```

Добавление коммитов из ветки к текущей:

```
...  
$ git merge <имя_ветки>  
...
```

Добавление произвольного коммита к текущей ветке:

```
...  
$ git cherry-pick <коммит>  
...
```

Будьте очень осторожны при совместном использовании merge и rebase (не делайте перебазир


```
```  

$ git reset --hard HEAD^
```
```

--- <h1 id="#работа-с-удалённым-репозиторием">Работа с удалённым репозиторием</h1>

* Репозиторий, от которого клонирован рабочий репозиторий, по умолчанию записан в рабочий репозиторий. Чтобы получить обновления, они будут доступны как ветки с именами origin/<имя_ветки_в_удалённом_репозитории>

```
```  

$ git fetch [origin]
```
```

Создать в удалённом репозитории рабочую ветку с тем же содержанием, что и текущая, и связать её с текущей веткой:

```
```  

$ git push -u origin <имя_ветки_в_origin>
```
```

Отправить изменения в ветку в удалённом репозитории, с которой связана текущая ветка:

```
```  

$ git push
```
```

--- <h1 id="#метки">Метки</h1>

Коммиты могут быть помечены метками. Метка – это заданное пользователем имя, под которым можно будет обратиться к конкретному коммиту.

Показать список меток:

```
```  

$ git tag
```
```

Создать метку (имя метки это имя файла, оно не должно содержать пробелов и спецсимволов)

```
```  

$ git tag <имя_метки>
```
```

Удалить метку:

```
```  

$ git tag -d <имя_метки>
```
```

<h1 id="#история-изменений">История изменений</h1>

Посмотреть историю последних значений HEAD (и команд, после которых эти значения были по

```
```  

$ git reflog
```
```

Комментарии